

16

# Lecture 16

**Deadlocks I: Characterization and Conditions**

By Brijesh Shukla  
C07: Operating Systems

**Join the lecture online on your dashboard**

# Today's Agenda

## Part I — Foundation

-  Introduction to deadlocks
-  Real-world analogies & motivation
-  System model & resource concepts

## Part II — Theoretical Framework

-  Deadlock characterization
-  Four necessary (Coffman) conditions
-  Deep dive into each condition

## Part III — Visual Analysis

-  Resource Allocation Graphs (RAGs)
-  RAG construction & interpretation
-  Examples of increasing complexity

## Part IV — Application

-  Classic deadlock problems
-  Real-world case studies
-  System impacts & consequences



# Let's Start with a Challenge!

## Deadlock Analogy: Narrow Bridge

 **One-lane bridge, two cars arrive together**

Both drivers refuse to reverse. ?

### What happens?

-  No movement
-  Stuck situation
-  Someone must give way
-  A rule is required

### Bridge to Computing

Same in computer systems: **Processes competing for resources can deadlock — no progress.**



# What is a Deadlock?

## Definition

A state where processes wait on each other indefinitely, so **no progress is possible**.

## Simplified

Blocked processes holding resources while waiting for others.

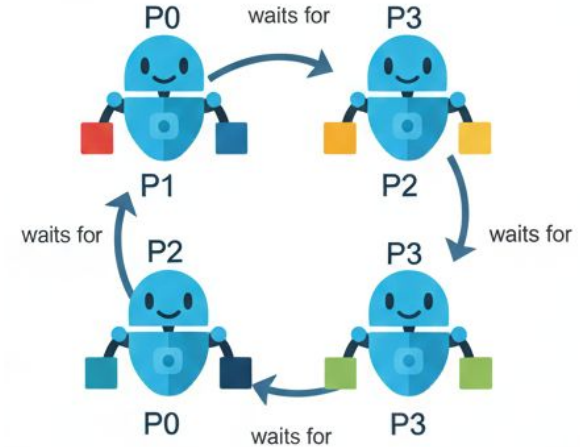
## Key Characteristics

-  Permanent block (no timeout)
-  Circular dependency
-  No self-recovery
-  Affects all involved processes

## Formal View

- Each process waits for another
- Circular wait exists:

$$P_0 \rightarrow P_1 \rightarrow \dots \rightarrow P_n \rightarrow P_0$$



# Deadlock vs Other Blocking States

## Starvation vs Livelock vs Deadlock

### Starvation

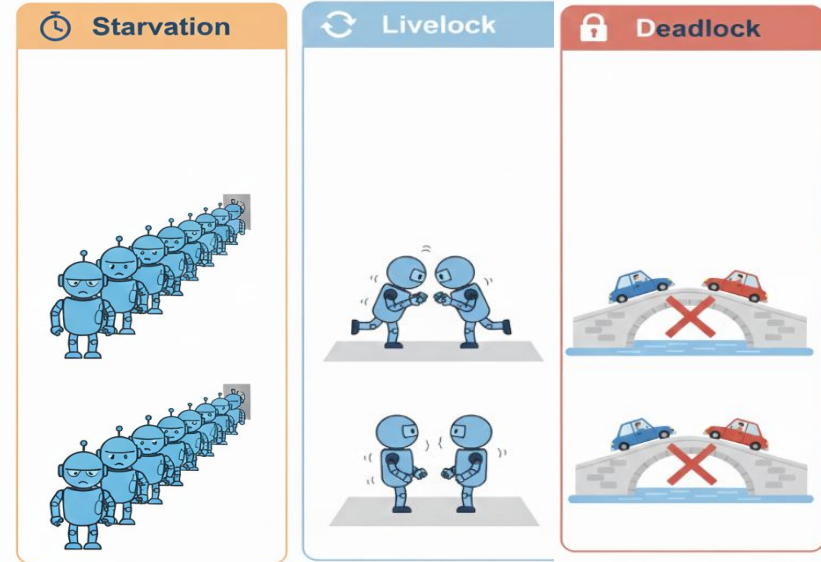
- Low-priority process waits
- Others make progress
- May resolve later

### Livelock

- State keeps changing
- No real progress
- Overreaction loop

### Deadlock

- Total standstill
- No state changes
- Progress impossible



## Simple Analogies

- **Starvation:** Queue that never reaches you
- **Livelock:** Two people sidestepping endlessly
- **Deadlock:** Cars stuck on a one-lane road

# System Model Resources

## Resource Types

### Reusable Resources

- Used by one process at a time
- Released after use
- CPU, memory, I/O, files

### Consumable Resources

- Created & destroyed
- Used once
- Signals, interrupts, messages

### Resource Instances

- Multiple identical units
- Allocated independently
- e.g., 4 CPUs, 2 printers



# Process-Resource Interaction

## Standard Resource Usage Pattern



### Request

- Ask for resource
- Wait if unavailable



### Use

- Access resource
- Critical section



### Release

- Free resource
- Allow others to proceed

## System Call Examples (Unix/Linux)



**Request:** `open()`, `malloc()`, `wait()`



**Use:** `read()`, `write()`, memory access



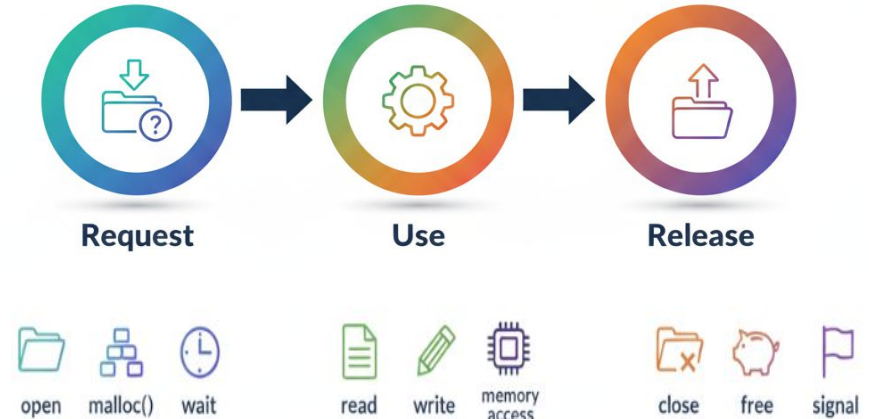
**Release:** `close()`, `free()`, `signal()`



### Important

Request → Use → Release is mandatory.

Failure to release ⇒ **starvation or deadlock**.



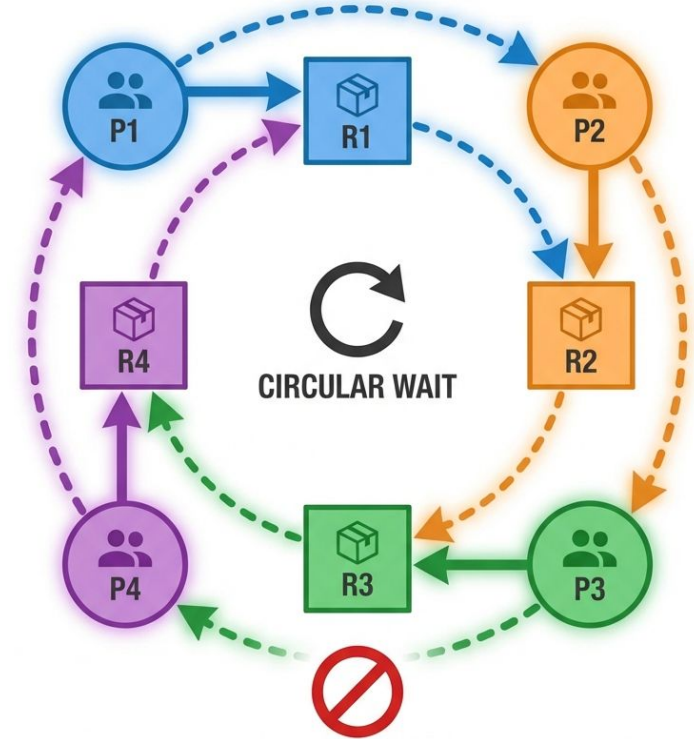
# Deadlock Characterization

## Definition

A set of processes is in **deadlock** when each process waits for an event that only another process in the set can cause.

## Key Characteristics

-  At least **2 processes**
-  At least **2 resources**
-  **Circular wait**
-  No independent progress



# Necessary Conditions (Coffman Conditions)

Deadlock occurs **if all four hold simultaneously**

## **Mutual Exclusion**

- Resource is non-shareable
- Only one process uses it

## **Hold & Wait**

- Holding one resource
- Waiting for another

## **No Preemption**

- Resources not forcibly taken
- Released voluntarily

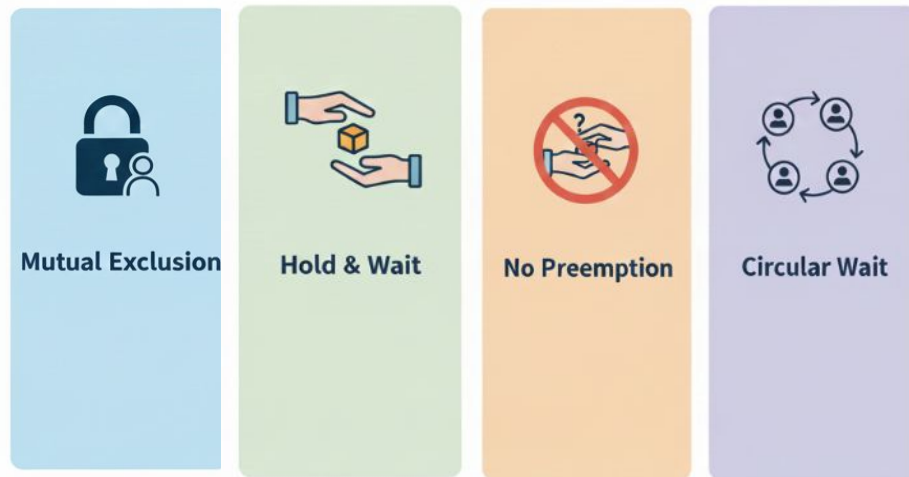
## **Circular Wait**

- Closed chain of processes
- Each waits for the next

## **Critical Insight**

These conditions are **necessary but not sufficient**

→ All must exist, but their presence alone doesn't guarantee deadlock.



# Condition 1 Mutual Exclusion

## Definition

Only **one process** can use a resource at a time. Others must **wait** until it's released.

## Why It Exists

- Non-shareable resources (e.g., printer)
- Data consistency (file writes)
- Hardware limits

## Examples

### ✓ Shareable (No Mutual Exclusion)

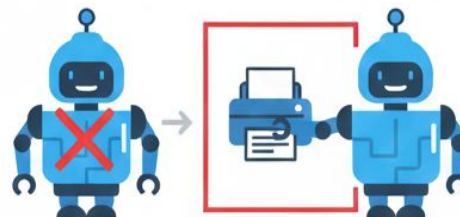
- Read-only files, Read-only memory and Reentrant code

### ✗ Non-Sharable (Needs Mutual Exclusion)

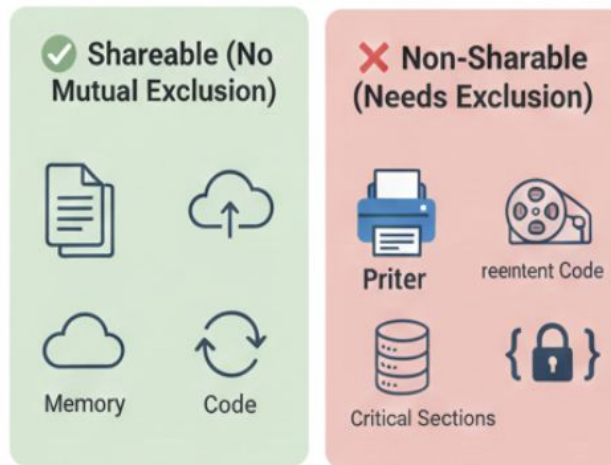
- Printers, scanners, Tape drives, Writable database records and Critical sections (mutex)

Code Example: Mutex (C / POSIX)

```
pthread_mutex_t lock;
pthread_mutex_lock(&lock); // Acquire
access_shared_resource(); // Critical section
pthread_mutex_unlock(&lock); // Release
```



Critical Section



# Code Example – Mutual Exclusion with Mutex

## Mutual Exclusion with Mutex

### ✗ Without Mutual Exclusion

- Race condition
- Lost updates
- Counter < **200000**

### ✓ With Mutual Exclusion

- One thread at a time
- Correct result: **200000**
- ⚠ Risk of deadlock (multiple mutexes)

| INPUT | OUTPUT                | ERROR |
|-------|-----------------------|-------|
| 1     | Final counter: 200000 |       |
| 2     |                       |       |

# Condition 2 Hold and Wait

## Definition

A process **holds at least one resource** while **waiting for others** held by different processes.

## Scenario

- P1 holds R1
- P1 requests R2 (held by P2)
- P1 keeps R1 while waiting

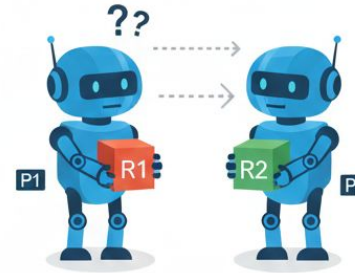
## Real Example



### Text Editor

- Holds: Keyboard buffer (R1)
- Waiting for: Printer (R2)
- Keyboard not released while waiting

## Scenario



- P1 waits for R2

## Real Example



### Text Editor

Holds: Keyboard buffer (R1)

Waiting for:  
Printer (R2)



Keyboard not released while waiting



## Key Insight

Hold-and-wait increases resource utilization,  
but combined with other conditions → **deadlock risk**.

# Code Example – Hold and Wait Pattern

## Hold and Wait Pattern

### Output: Hold & Wait Deadlock

-  **Process A** holds **R1**, waits for **R2**
-  **Process B** holds **R2**, waits for **R1**
-  Resources not released while waiting
-  Circular wait formed

| INPUT | OUTPUT                                                     | ERROR |
|-------|------------------------------------------------------------|-------|
| 1     | Process A: Requesting Resource 1...                        |       |
| 2     | Process A: Got Resource 1, now holding it                  |       |
| 3     | Process A: Now requesting Resource 2 (while holding R1)... |       |
| 4     | Process A: Got both resources!                             |       |
| 5     | Process A: Using both resources                            |       |
| 6     | Process B: Requesting Resource 2...                        |       |
| 7     | Process B: Got Resource 2, now holding it                  |       |
| 8     | Process B: Now requesting Resource 1 (while holding R2)... |       |
| 9     | Process B: Got both resources!                             |       |
| 10    | Process B: Using both resources                            |       |

# Condition 3 No Preemption

## Definition

Resources **cannot be forcibly taken**. They are released **only voluntarily** by the holding process.

## Why It Matters

- Data integrity
- Prevents corruption
- Consistent process state

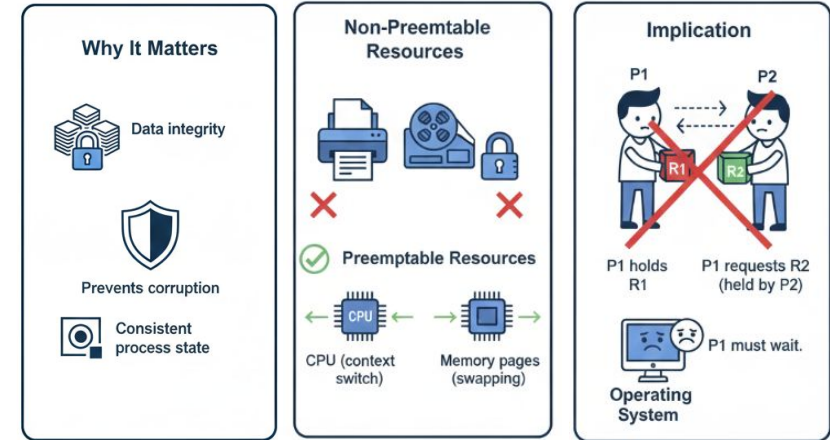
## Examples

### ✗ Non-Preemptable Resources

- Printer (mid-print)
- Tape drive (write)
- Database locks (transaction)

### ✓ Preemptable Resources

- CPU (context switch)
- Memory pages (swapping)



## 🔑 Key Insight

No preemption preserves correctness, but contributes to **deadlock** when combined with other conditions.

# Code Example – Non-Preemptible Mutex

## Non-Preemptible Mutex

### Output: No Preemption Demonstrated

-  **High-priority process** acquires resource first
-  **Low-priority process** requests urgently
-  Cannot preempt the holder
-  Must wait until release

| INPUT | OUTPUT                          | ERROR |
|-------|---------------------------------|-------|
| 1     | Task A: Resource acquired       |       |
| 2     | Task B: Waiting for resource... |       |
| 3     | Task A: Resource released       |       |
| 4     | Task B: Resource acquired       |       |
| 5     |                                 |       |

# Condition 4 Circular Wait

## Definition

A **closed chain** of processes exists where each process waits for a resource held by the next.

## Formal View

$P_0 \rightarrow P_1 \rightarrow \dots \rightarrow P_n \rightarrow P_0$

( $P_i$  waits for a resource held by  $P_{i+1}$ )

## Example Scenario

- **P1** holds **R1**, waits for **R2**
  - **P2** holds **R2**, waits for **R3**
  - **P3** holds **R3**, waits for **R1**
- **P1** → **P2** → **P3** → **P1**

## Deadlock-Prone Code

// Process P1

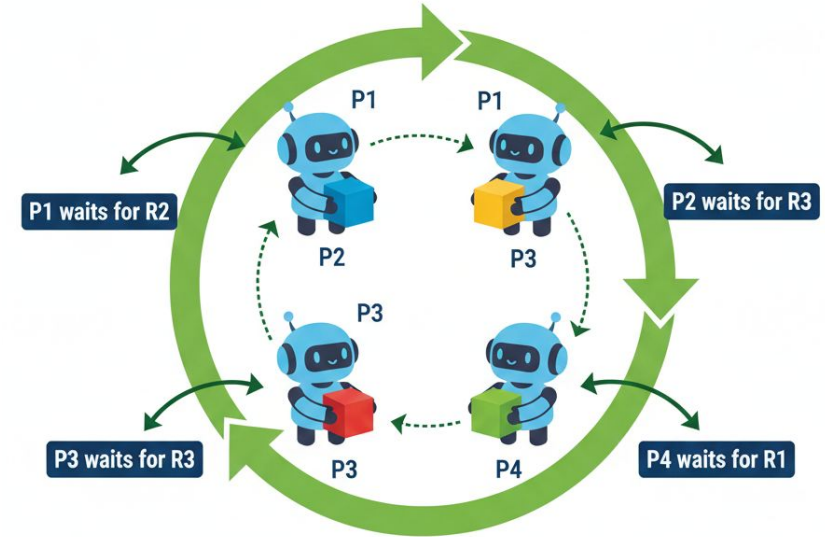
**acquire**(**R1**);

**acquire**(**R2**); // waits if P2 holds R2

// Process P2

**acquire**(**R2**);

**acquire**(**R1**); // waits if P1 holds R1



## Key Insight

**Circular wait is the final condition** that completes deadlock.  
Breaking the cycle ⇒ **deadlock prevention**.

# Code Example – Three-Process Circular Wait

## Three-Process Circular Wait

### Circular wait chain:

- P1 holds R1, wants R2
- P2 holds R2, wants R3
- P3 holds R3, wants R1

**Cycle: P1 → R2 → P2 → R3 → P3 → R1 → P1**

| INPUT | OUTPUT                                             | ERROR |
|-------|----------------------------------------------------|-------|
| 1     | P1: Locking R1                                     |       |
| 2     | P2: Locking R2                                     |       |
| 3     | P3: Locking R3                                     |       |
| 4     | P2: Trying to lock R3...                           |       |
| 5     | P1: Trying to lock R2...                           |       |
| 6     | P1: DEADLOCK detected (R2 held by another process) |       |
| 7     | P2: DEADLOCK detected (R3 held by another process) |       |
| 8     | P3: Trying to lock R1...                           |       |
| 9     |                                                    |       |

# Resource Allocation Graph (RAG) Introduction

## Purpose

A **directed graph** that visually represents **resource allocation and deadlock** situations.

## Graph Components

### Vertices

**Processes (P) – Circles**

- $P = \{P_1, P_2, \dots, P_n\}$

**Resources (R) – Rectangles**

- $R = \{R_1, R_2, \dots, R_m\}$
- Dots inside = resource instances

## Edges

**Request Edge:**  $P_i \rightarrow R_j$

- Process  $P_i$  is waiting for  $R_j$

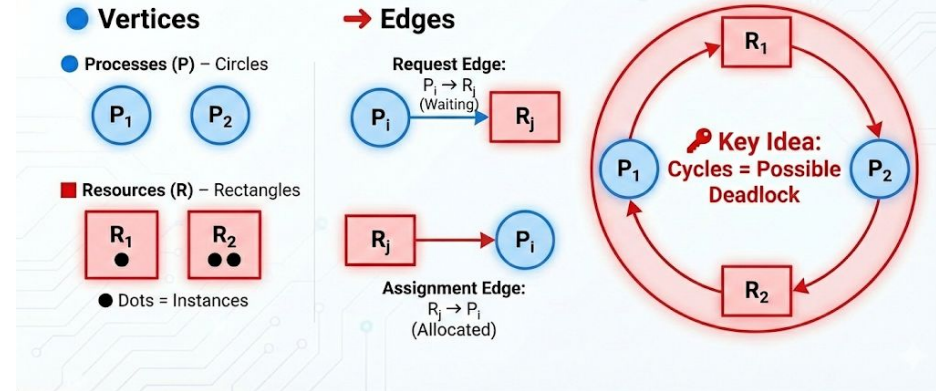
**Assignment Edge:**  $R_j \rightarrow P_i$

- Resource  $R_j$  allocated to  $P_i$

## Key Idea

Cycles in the graph indicate **possible or actual deadlock**.

## Resource Allocation Graph (RAG) Introduction



# RAG Notation and Symbols

## Symbols

- ○ **Process**:  $P_1, P_2, \dots$
- □ **Resource**:  $R_1, R_2, \dots$
- • **Dots**: resource instances

## Edges

- → **Request**:  $P_i \rightarrow R_j$  (process waiting)
- ← **Assignment**:  $R_j \rightarrow P_i$  (resource held)

## Example

- $P_1 \rightarrow R_1$  :  $P_1$  requests  $R_1$
- $R_2 \rightarrow P_3$  :  $R_2$  allocated to  $P_3$

## How to Read the Graph

- Edge **into process**  $\rightarrow$  resource held
- Edge **out of process**  $\rightarrow$  waiting for resource
- Edges **from resource**  $\rightarrow$  allocated instances
- Edges **to resource**  $\rightarrow$  waiting requests

## Important Notes

- Request  $\rightarrow$  Assignment when granted
- Assignment removed on release
- Graph updates **dynamically**

# RAG Example 1 No Deadlock

## Setup

- **Processes:**  $P_1, P_2, P_3$
- **Resources:**
  - $R_1$  (1 instance)
  - $R_2$  (2 instances)
  - $R_3$  (1 instance)

## Allocations & Requests

- $R_1 \rightarrow P_1$
- $R_2 \rightarrow P_1, R_2 \rightarrow P_2$
- $P_2 \rightarrow R_3$
- $R_3 \rightarrow P_3$

## Analysis

-  No cycle in graph
-   $P_3$  can finish first
-  Releases  $R_3 \rightarrow P_2$  proceeds
-   $P_1$  completes afterward

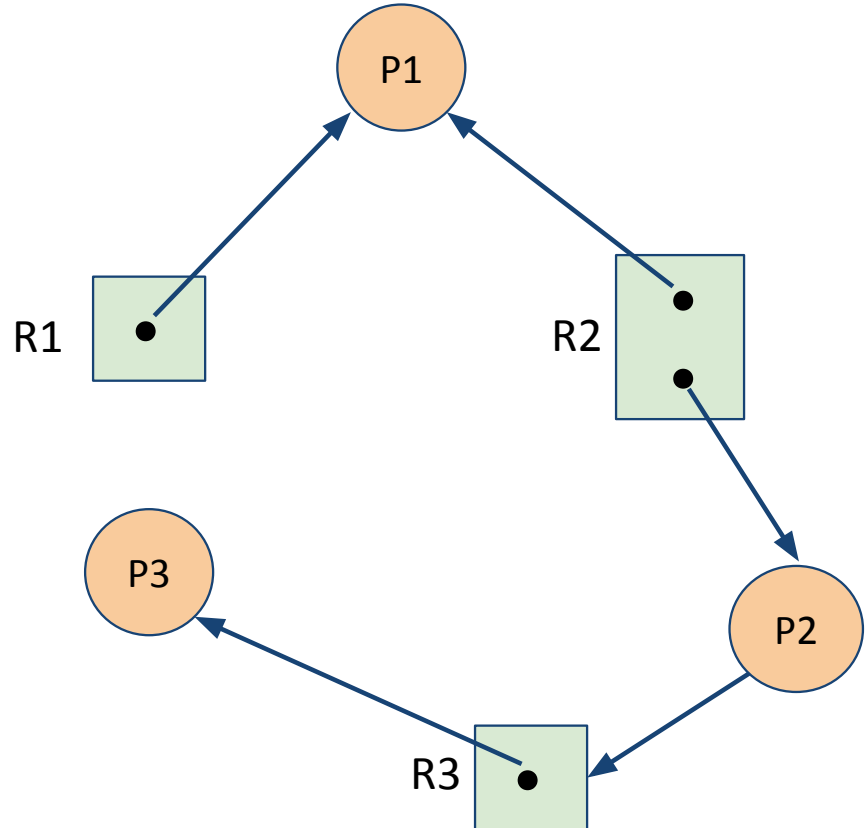
## Safe Execution Order

$P_3 \rightarrow P_2 \rightarrow P_1$

## Conclusion

 **NO DEADLOCK**

System is in a **safe state**.



# RAG Example 2 Deadlock Exists

## Setup

- **Processes:**  $P_1, P_2, P_3$
- **Resources:**  $R_1, R_2, R_3$  (1 instance each)

## Allocations & Requests

- $R_1 \rightarrow P_1, P_1 \rightarrow R_2$
- $R_2 \rightarrow P_2, P_2 \rightarrow R_3$
- $R_3 \rightarrow P_3, P_3 \rightarrow R_1$

## Analysis

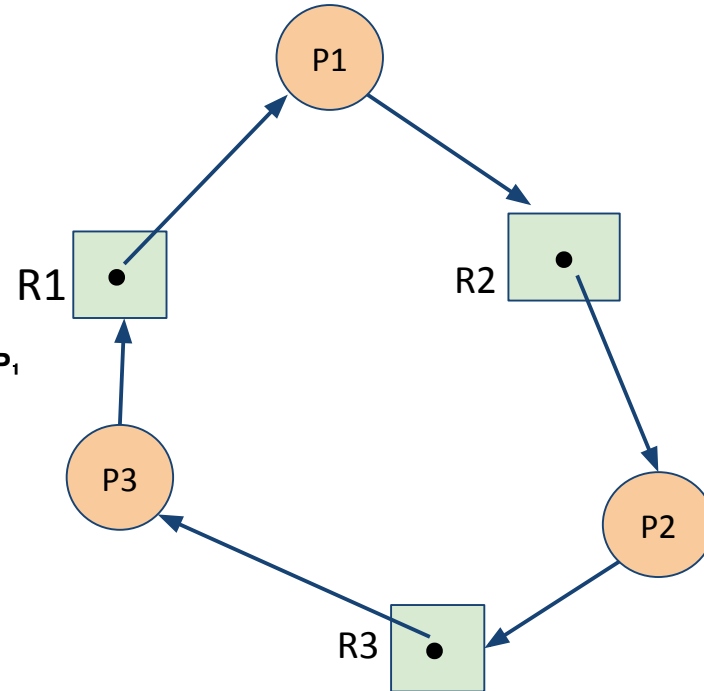
-  **Cycle detected:**  $P_1 \rightarrow R_2 \rightarrow P_2 \rightarrow R_3 \rightarrow P_3 \rightarrow R_1 \rightarrow P_1$
- Single instance per resource
- All processes blocked

## Why Deadlock Occurs

- $P_1$  holds  $R_1 \rightarrow$  needs  $R_2$
- $P_2$  holds  $R_2 \rightarrow$  needs  $R_3$
- $P_3$  holds  $R_3 \rightarrow$  needs  $R_1$
-  **Circular wait satisfied**

## Conclusion

 **DEADLOCK EXISTS:** No process can proceed.



# RAG Example 3 Cycle But No Deadlock

## Setup

- **Processes:**  $P_1, P_2, P_3$
- **Resources:**  $R_1$  (2 instances),  $R_2$  (1 instance)

## Allocations & Requests

- $R_1 \rightarrow P_1, R_1 \rightarrow P_3$
- $P_1 \rightarrow R_2$
- $R_2 \rightarrow P_2$
- $P_2 \rightarrow R_1$
- $P_3$  can finish

## Draw RAG by Yourself

### Analysis

-  Cycle exists:  $P_1 \rightarrow R_2 \rightarrow P_2 \rightarrow R_1 \rightarrow P_1$
-   $R_1$  has **multiple instances**
-   $P_3$  can release  $R_1$
-  Others proceed sequentially

## Safe Execution

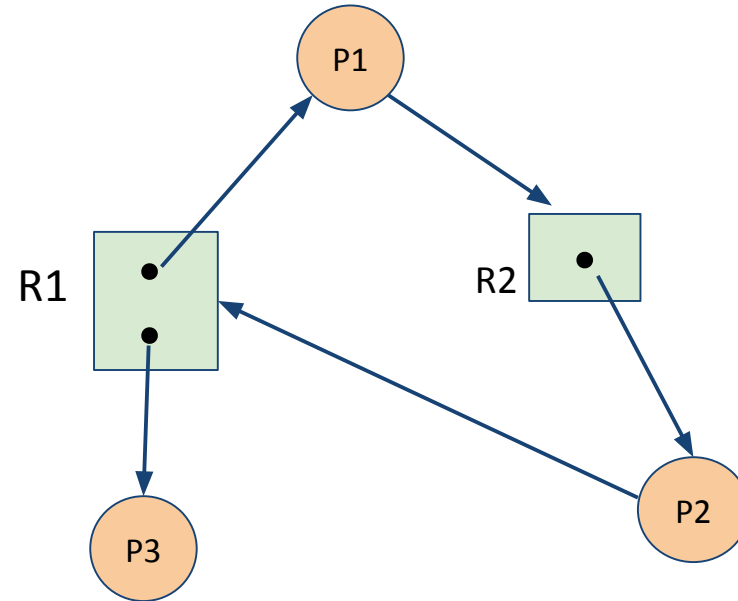
$P_3 \rightarrow P_2 \rightarrow P_1$

## Conclusion

 **NO DEADLOCK** (despite cycle)

## Key Insight

With **multiple resource instances**, a cycle is **necessary but not sufficient** for deadlock.



# RAG Analysis Summary Rules

## Case 1: No Cycle

### ✓ No Deadlock

- Safe state
- All processes can finish

## Case 2: Cycle Exists

### ♦ Single Instance per Resource

### ✗ Deadlock Exists

- All processes in cycle blocked

### ♦ Multiple Resource Instances

### ⚠ Deadlock May Exist

- Further analysis needed
- Check resource release outside cycle
- Use **Banker's / detection algorithms**

## Key Rule

🔑 Cycle is necessary for deadlock

🔑 Cycle is sufficient only if each resource has one instance

# Deadlock Example Dining Philosophers

## Problem Setup

- 5 philosophers, circular table
- 5 chopsticks (one between each pair)
- Each philosopher needs **2 chopsticks**
- Alternates between **thinking** and **eating**

## Deadlock Scenario

1. All pick up **left chopstick**
2. All try to pick up **right chopstick**
3. Right chopstick held by neighbor
4. Everyone waits forever

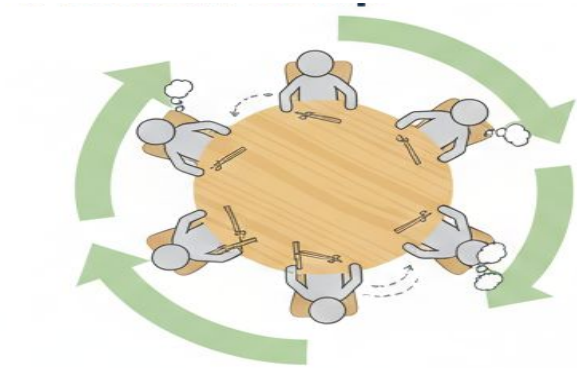
➡ **DEADLOCK**

## Why Deadlock Occurs (All 4 Conditions)

-  **Mutual Exclusion:** One chopstick → one philosopher
-  **Hold & Wait:** Holding left, waiting for right
-  **No Preemption:** Cannot force release
-  **Circular Wait:**  $P_1 \rightarrow P_2 \rightarrow \dots \rightarrow P_5 \rightarrow P_1$

## 🔑 Key Insight

Dining Philosophers is a **classic deadlock illustration** showing how all four conditions can arise naturally.



## Deadlock Scenario



# Consequences of Deadlocks

## Resource Wastage

- Wasted CPU cycles
- Memory held unnecessarily

## System Performance

- Lower throughput
- Higher response time

## Application Failures

- Incomplete transactions
- Possible data inconsistency

## Economic Costs

- Productivity loss
- Downtime expenses

## Real-World Impact

 **Databases:** 1–5% transactions deadlock

 **Operating Systems:** May freeze completely

 **Distributed Systems:** Hard to detect & resolve

## Resource Wastage



## System Performance



## Application Failures



# Necessary vs Sufficient Conditions

**All 4 Coffman Conditions are necessary:**

- Mutual Exclusion, Hold and Wait, No Preemption and Circular Wait

→ If any one is missing → **NO** deadlock

## Sufficient Conditions

✓ Presence **guarantees** deadlock

Deadlock is **certain** when:

- All 4 conditions present
- Each resource has **one instance**
- **Cycle exists** in RAG

## Key Distinction

- **Necessary** → “Must have”
- **Sufficient** → “Guarantees”

✗ All 4 present ≠ guaranteed deadlock

✓ All 4 + single instances + cycle = deadlock

## Practical Insight

🛑 Prevent deadlock by breaking at least **ONE** necessary condition



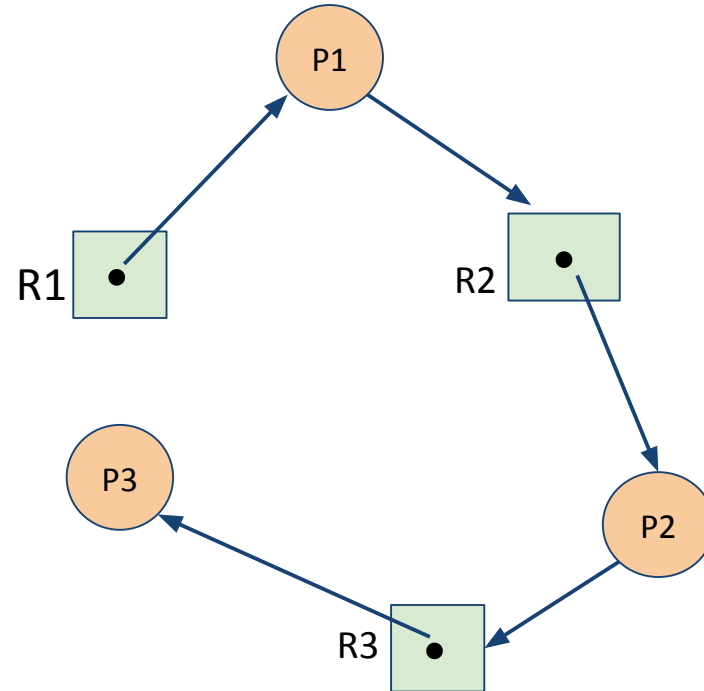
# Practice Problems

## Problem 1: Given:

- P1 holds R1  $\rightarrow$  requests R2
- P2 holds R2  $\rightarrow$  requests R3
- P3 holds R3
- All resources: **1 instance**

## Result:

- ✗ **No Deadlock**
- ➡ No cycle (P3 not waiting)



# Practice Problems

## Problem 2: Given:

- Processes: P1–P4
- Resources: R1(2), R2(1), R3(3)
- P1: holds R1 → requests R2
- P2: holds R2 → requests R3
- P3: holds 2×R3 → requests R1
- P4: holds 1×R3

## Analysis:

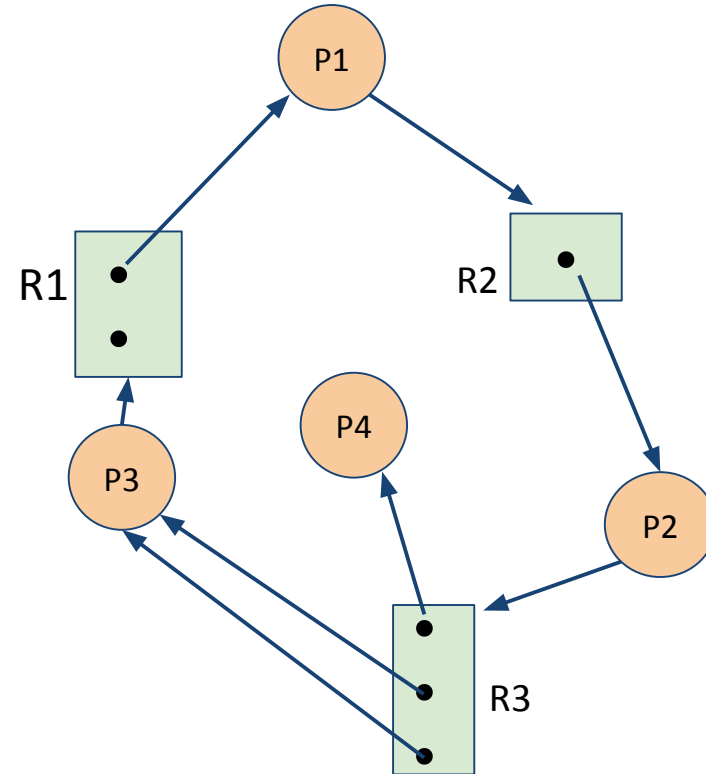
- Cycle exists
- Multiple resource instances
- P4 / P3 can release resources

## Result:

✓ No Deadlock (Safe State)

## Key Tips:

- No cycle → No deadlock
- Cycle + single instance → Deadlock
- Cycle + multiple instances → Analyze further



# Key Takeaways

- Deadlock Definition
- Real-World Impact
- Four Necessary Conditions (Coffman Conditions)
- Resource Allocation Graphs (RAG)
- RAG Analysis



# Thank You!

Thank you for your attention. Feel free to ask questions or revisit the topics covered.